2012 IEEE 30th VLSI Test Symposium (VTS)

Ping-Pong Test: Compact Test Vector Generation for Reversible Circuits

Masoud Zamani
Dept. Electrical and Computer Eng.
Northeastern University
Boston, MA, USA
Email: mzamani@ece.neu.edu

Mehdi B. Tahoori
Faculty of Informatics
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: mehdi.tahoori@kit.edu

Krishnendu Chakrabarty
Dept. Electrical and Computer Eng.
Duke University
Durham, NC, USA
Email: krish@ee.duke.edu

***Abstract*—Reversibility as an inherent requirement of quantum computation motivates further research on reversible logic. Due to anticipated high failure rates for such technologies, thorough testing is a must for these circuits. In this paper, we present a compact test generation and application method for reversible circuits which achieves high (100%) fault coverage and can be adopted for BIST implementations. In this method, the next test pattern is the response of the reversible circuit to the previous test pattern. A test generation algorithm to minimize test time and achieve 100% fault coverage is also presented. Simulation results on a set of reversible benchmark circuits confirm that this approach can detect all single missing/repeated gate faults as well as the majority of multiple faults.**

I. Introduction

Studies on reversible logic were initiated in 1985, based on a thermodynamic theory stating every lost or duplicated bit information causes energy loss of $kT \ln 2$, where k is the Boltzmann's constant and T is the temperature [1]. Therefore, reversible computation, which avoids the information loss, was introduced as a promising computational logic approach to reduce or even eliminate the power consumed by the computation. A circuit without fan-out branches and feedback loops is called reversible if each input pattern to the circuit can be determined from the output pattern and vice versa [2]. Further research on reversible logic is motivated based on the fact that reversible logic is an inherent design requirement of quantum computation [2], [3], [4]. Although, irreversible logic is used in designing circuits in current CMOS technology, however, it is gaining more attention recently, since quantum computers are reversible in nature and reversibility is a necessary condition for energy free computation in future technologies.

By scaling the technology, testing complexity of post-fabrication and error rate are increasing, which is expected to be worsen in future technologies. In particular, failures and dispersions are expected during quantum operations in quantum computers. Hence, efficient testing techniques to be used in the field are crucial for quantum computers as a realization of reversible circuits [3].

978-1-4673-1074-1/12/ $ 31.00 © 2012 IEEE

Full controllability and observability of reversible circuits simplifies test generation (especially to detect stuck-at fault) for reversible circuits, compared to irreversible circuits [5]. It has been shown that for any reversible circuit, there is a complete single stuck-at fault test set, which also can detect all multiple stuck-at faults in the reversible circuit [6]. However, relevance of stuck-at fault models for reversible circuits has been questioned [7]. Therefore, inspired from technological requirements of physical realization of reversible circuits, a set of appropriate logical fault models have been introduced for reversible circuits. These fault models mainly include *Missing Gate Fault* (MGF) and *Repeated Gate Fault* (RGF) models. In these models fault is characterized as elimination (replication) of gates in the circuit [8]. A set of DFT and ATPG techniques has been proposed to detect these faults [9], [10].

In this paper, we propose a so-called *Ping-Pong test* as a new testing technique for reversible circuits. The objective of Ping-Pong test is reducing the information required for high (100%) fault coverage testing. In Ping-Pong testing, a test vector is applied to the circuit and the response of the circuit for that vector is used as the next test vector. The test vector set can be represented only by the initial vector and the length of the sequence, which can drastically reduce test data volume. This compact test generation technique can be used for efficient BIST implementation for reversible circuits. To keep the test time comparable to traditional test, a test sequence generation method is proposed to minimize the number of applied patterns to achieve the required coverage. We have shown an implementation for 100% fault coverage of single MGF and RGF and the flow can be extended to multiple faults as well as traditional fault models such as stuck-at faults.

The organization of the rest of the paper is as follows. Section II discusses preliminaries, fault models for reversible circuits and related work. The proposed testing approach is introduced in Section III. An implantation flow for Ping-Pong test is presented in Section IV. Experimental results are shown in Section V. Finally, Section VI concludes the paper.

II. Preliminaries and Related Work

A. Reversible Circuit

A bijective logic function $f : X \rightarrow Y$ from the set of inputs $X = \{x_1, x_2, ..., x_n\}$ to the set of outputs $Y = \{y_1, y_2, ..., y_m\}$ is a reversible function iff [11]:

1) The number of inputs of the function is equal to the number of its outputs (i.e. $n = m$).
2) That function is a one-to-one relation between the input permutations and the output permutations.

Inputs of reversible gates are divided to two groups: *Target* inputs (T) and *Control* inputs (C). Outputs are also categorized as *Garbage* outputs (G) and *Function* outputs (F). Control inputs control logic operation of function outputs on target inputs. Garbage outputs are unused outputs which are added due to reversibility. Commonly studied reversible gates include: NOT, *Feynman Gate*, and *Toffoli Gate* [12], [13], [14]. Fig. 1 shows these gates. These gates are also known as *K-Controlled-NOT (K-CNOT)*, where K denotes number of control inputs. In a K-CNOT gate $F = T \oplus \prod_{i=1}^{K} C_i$ and for each i, $G_i = C_i$. It must be noted, any irreversible function can be transformed to a reversible circuit using reversible gates, with adjustments on branches and feedback loops. However, reversible implementation may result in some area overhead.

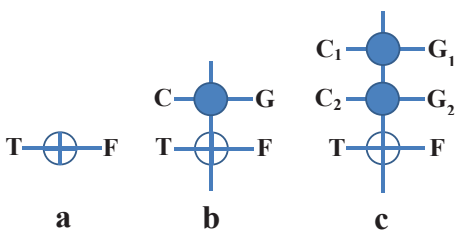


Fig. 1. Reversible gates: (a) NOT (b) Feynman Gate (c) Toffoli Gate

B. Logical Fault Models for Reversible Circuits

The requirements of a technology to implement quantum computers as a realization of reversible circuits have been discussed in [15], [7]. Based on such requirements, trapped-ion technology is considered as a promising candidate to implement quantum computers as well as reversible circuits [15], [7], [16]. A set of appropriate logical fault models for reversible circuits have been proposed [7]. These fault models, mainly inspired from ion-trap quantum computing technologies, are as follows.

- *Single Missing Gate Fault* (SMGF); In this fault, a single gate is completely removed from the circuit [7]. Fig. 2.b shows the corresponding circuit when there is a SMGF at the second gate.
- *Repeated Gate Fault* (RGF); In this fault, a gate is replaced by several instances of the same gate [7]. Fig. 2.c shows the case in which second gate is repeated. If a gate is replaced by even number of instances of the same gate, then fault effect is identical to SMGF of that gate [7].
- *Multiple Missing Gate Fault* (MMGF); In this model, it is assumed that several consecutive gates are completely removed from the circuit (Fig. 2.d). In contrast to traditional definition of multiple faults, MMGF is not necessarily the same as multiple SMGF. In this model, all of the missed gates must be consecutive gates, while multiple SMGF implies that several distinct arbitrary gates are missed which are not necessarily consecutive [7].
- *Partial Missing Gate Fault* (PMGF); This fault, also known as a disappearance fault, assumes a subset of control inputs of a gate is removed. This fault turns a K-CNOT gate into K'-CNOT gate, where $K' < K$. If m of the K control inputs of a K-$CNOT$ is removed, then the gate will be transferred to $(K-m)$-$CNOT$ gate, where m is called the order of PMGF [17]. Fig. 2.e shows one of the possible PMGFs on the first gate.

C. Related Work

A DFT method to detect SMGFs in a reversible circuit is introduced in [8], which is done by adding some extra gates. An appearance fault on the added gates results in incorrect functionality of the circuit. A test pattern generation method to detect all MMGFs has been introduced in [10]. Although, the introduce method is an optimal method for test set generation, the detection of all MMGFs requires a large number of test vectors.

Previously proposed test compaction techniques use compression (encoding) methods to reduce test data volume by decompressing (decoding) the encoded test data (e.g. [18]). In [19], using Golomb coding, the response of the fault-free circuit for each test pattern is used in compression of the next test pattern. However, these techniques result in both area overhead (decoder) and test time increase (decompression time). In contrast, our proposed method needs to store only very few test vectors without any need for encoding or decoding. Using response of a circuit to a test pattern for generating the next test pattern in conventional logic has been studied in literatures (e.g. [20], [21], [22]). Such method is used in a type of BIST architecture for CMOS circuits, called Circular BIST and *Circuit Self-Test Path* (CSTP). The major challenge of using such BIST architecture in conventional logic is low fault coverage due to limited cycles, correlations in test patterns, and lake of randomness in test patterns [22]. In this paper, we introduce an implementation of CSTP in reversible circuits, where the outputs of the circuit are directly used as the inputs, without any modifications in terms of shift or XOR, in contrast to CSTP in conventional circuits. We show that the direct use of the outputs is sufficient to achieve acceptable fault coverage. Furthermore, we propose an algorithm to select output sets, such that 100% fault coverage is achieved by very few test cycles.

III. Ping-Pong Test

The objective of *Ping-Pong test* is to derive the minimum amount of information required for a test set to achieve full coverage for testing reversible circuits. Without loss of generality, we assume single SMGFs/RGFs as the target fault model. However, with slight modifications, the method can be extended to other fault models.

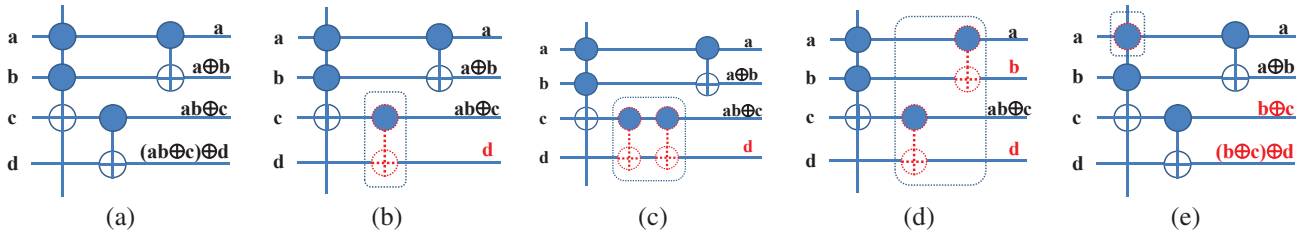
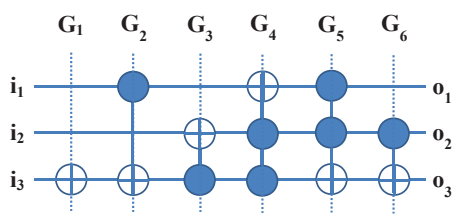


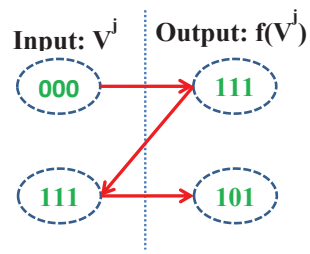
Fig. 2. Illustration of logical fault models for reversible circuits: (a) the fault-free circuit (b) the SMGF on the second gate (c) the RGF on the second gate (d) the MMGF of the last two consecutive gates (e) the first-order PMGF of the first control input of the first gate



(a)

Input	Output	Coverage
$i_1\ i_2\ i_3$	$o_1\ o_2\ o_3$	Gates (G)
0 0 0	1 1 1	1,3,4,5,6
0 0 1	0 0 0	1
0 1 0	0 0 1	1,3
0 1 1	0 1 1	1,6
1 0 0	1 0 0	1,2
1 0 1	1 0 1	1,2,3,4,6
1 1 0	1 1 0	1,2,5,6
1 1 1	1 0 1	1,2,3

(b)



(c)

Fig. 3. An example of Ping-Pong test vector: (a) the schematic of a circuit (Circuit 3_17 [23]) (b) the truth table and the fault coverage of each input pattern (c) a Ping-Pong sequence to detect all faults ($V^1 = 000,\ V^2 = 111$); $T = (000, 2, 101)$

A reversible circuit can be expressed as a bijective function over a set of inputs. For example, operation of a reversible circuit on an input pattern V^m; ($V^m = \{i_1^m, i_2^m, ..., i_n^m\}$) is expressed as $f(V^m)$; $f(V^m) = \{o_1^m, o_2^m, ..., o_n^m\}$. In a Ping-Pong test, the subsequent test vector is the response of the circuit to the previous test vector, i.e. in a sequence of k test vectors (V^1, V^2, ..., V^k), $V^j = f(V^{j-1})$; $\forall j; 2 \leq j \leq k$. This can greatly reduce the amount of test data volume. If there is such test vector sequence, then instead of storing all of the vectors, testing can be identified by three values: i- the first test vector (V^1), ii- the number of required vectors (k), which indicates the number of test cycles, and iii- the expected output pattern for the last test vector in the sequence ($f(V^k)$). So, instead of storing $2kn$ bits for test vectors and responses, only $2n + \log k$ bits of information is enough. A Ping-Pong test is defined as $(V^1, k, f(V^k))$. It is obvious that all these k test vectors in a Ping-Pong test can be ordered in a unique sequence, i.e., there is no repetitions, which is implied from reversibility. Therefore, for each reversible circuit, a set of Ping-Pong test vectors is identifiable. Ideally one Ping-Pong test can be used to cover all faults. However, in order to reduce test time (k) and improve coverage, various Ping-Pong test *sessions* might be needed $\{V_i^1, k_i, f(V_i^1)\}$, $i = 1$ to N. Therefore, efficient test generation algorithms are needed to minimize test time, $\sum_i k_i$ and test data volume (N).

An example of a sequence of input vectors which has the Ping-Pong property is shown in Fig. 3. Ping-Pong test can be identified as $T = (V^1, k, f(V^k))$, where, T is the required test information. This information can be translated as applying a sequence of k test vectors (k cycles), starting from V^1 ending to final output $f(V^k)$. For example, $T = (000, 2, 101)$ shows the Ping-Pong test of the circuit in Fig. 3. The fault coverage of a sequence of test vectors is less than or equal to the fault coverage of each individual test vectors. In other words, if a fault is not detected by none of V^1, V^2, ..., and V^k test vectors, then that fault is not detected by a sequence constructed by these vectors.

IV. Test Sequence Generation

The concept of Ping-Pong test is general and can be applied to various fault models. As mentioned before, the purpose of Ping-Pong test sequence generation is to minimize test time (total number of cycles) and test information (the number of Ping-Pong test sessions) while achieving the required coverage for a given fault model. Although feedback loops are not allowed in reversible circuits, the circuitry to apply Ping-Pong test acts as an environment around the circuit, and hence, it does not affect the reversibility of the circuit.

In this section, we focus on SMGF and single RGF as the fault model to show how the proposed method works. We use a set of heuristic and greedy algorithms to find required test sequences of Ping-Pong test of an arbitrary reversible circuit. This implementation consists of four major steps: 1- Finding the coverage list for each fault, 2- Searching for possible sequences, 3- Fault simulation and cycle dropping, and 4- Optimal sequence selection.

A. Finding the coverage list for each fault

In the first step, for each fault (i.e. gate as the fault site), we identify all of the possible input permutations which can detect

missing and as a result repeating of that gate. Using Lemma 1 a back-tracing method is used to identify the coverage list of each gate.

Lemma 1: An input pattern can detect a single SMGF or a single RGF on a gate if it justifies the logical values of 1 at all of the control inputs of that gate [8].

Based on Lemma 1, to detect a missing/repeated gate fault on a gate it is sufficient to apply 1 to all of the control inputs of that gate. However, the other lines contain arbitrary logical value (i.e., don't-care). Therefore, to find the coverage list of each gate at level i, its control inputs are set to 1, while the other inputs are set to a don't-care value (denoted by X). Then, the logical values of the inputs of the gate at level $i-1$ are identified, such that these values satisfy the inputs of the gate at level i. This process is back-traced to reach primary inputs. The values on primary inputs denote all possible input patterns to detect missing/repeated gate on gate i. As an example, Fig. 4 shows this process to identify coverage list of missing on gate G_3. By repeating this method for all gates, the coverage list of the gates in the circuit is determined. The runtime of finding coverage list of a fault is linearly proportional to the number of gates.

B. Search for possible sequences

Starting from each input pattern, there is a sequence of vectors T_n, where n is the decimal representation of the binary value of the first input pattern in the sequence (V^1). A vector V^i can be added to the sequence T_n if V^i is the output of the circuit for the last member of T^n (i.e. $V^i = f(V^{(i-1)})$, where V^{i-1} is the last added (the last enumerated) vector in the sequence. Since, there may be an arbitrary length for the sequences (number of cycles), the following constraints are applied during the process of adding a vector to a sequence:

1) The V^i can be added to the sequence T_n, if it is not equal to none of the previously added vectors to T_n. In other words, adding a vector which already has been added to T_n, creates a loop in the sequence of the Ping-Pong test.
2) A new vector can be added to the sequence T_n, if the number of vectors in T_n is less than a threshold. This threshold defines the maximum number of cycles in the Ping-Pong test session. In our implementations, the number of gates (stages) in the circuit is used as the threshold on the maximum number of cycles in each sequence of Ping-Pong test session.
3) The set of faults possibly detected by T_n is called *possible fault list* of the sequence T_n, and denoted by P^{T_n}. For each sequence, the possible detected fault list is created by the union of the sets of faults which are detected by each individual vectors of that sequence. A vector V^i can be added to the sequence T_n if $C^i \not\subset P^{T_n}$. The converge list of each input vector (C^i) is determined by searching in the coverage list of the gates. However, if vector V^i is added to the sequence T_n, then C^i is also added to P^{T_n}.

The runtime of searching for the possible sequences is exponentially proportional to the number of inputs (which is typically small) and linear to the number of gates.

C. Fault simulation and cycle dropping

After searching for the possible sequences, possible detected coverage list of sequence T_n (P^{T_n}) contains the faults which are potentially but not necessarily detected by T_n. This is due to the fact that faults which are detected by each individual test vectors they may be detected by a sequence containing those test vectors. However, since the final output of the sequence is checked, therefore there is a possibility the fault effect does not propagate to the final cycle of the sequence. Therefore, each sequence is simulated with respect to the fault in P^{T_n} to determine the actual C^T of the sequence. In other words, sequence T_n is simulated in faulty cases, where the faults are selected one by one from P^{T_n}. If there is a fault in P^{T_n} which is detected by T_n, then it is added to C^{T_n}. This step of the algorithm determines the actual fault coverage of each sequence.

Since, there is a possibility that the size of C^{T_n} is less than the size of P^{T_n}, therefore there may be some redundant cycles at the end of the sequence, which can be removed if it satisfies two conditions. A vector can be dropped if:

1) Removing the vector does not affect the actual fault coverage list of the sequence. Such a vector is called redundant vector.
2) The dropped vector is the last vector in the sequence. A vector in the middle of the sequence cannot be removed (even if it is redundant), because removing from the middle of the sequence brakes the sequence. Breaking a sequence may reduce coverage list of the sequence. It is due to the fact that vectors after the breaking point are not reachable; therefore the faults detected by them may be removed from the coverage list.

The runtime of fault simulation and cycle dropping is similar to the runtime of searching for the possible sequences, which is exponentially proportional to the number of inputs and linear to the number of gates.

D. Sequence selection

The optimal selection of the sequences can be converted to *Minimum Dominating Set Bipartite* problem, which is a classical problem in graph theory [24]. Therefore, a bipartite graph is constructed from the sequences and the fault list. The nodes in one side of the graph represent the sequences, and the nodes in the other side represent all possible single faults (equal to the number of gates). There is an edge between a node of sequences and a node of faults, if the corresponding sequence can detect that fault. Each sequence node has a cost equal to the number of its cycles over the number of faults detected by that sequence. Now the problem is selecting the minimum number of sequence nodes which results in minimum number of sum of the costs, while the selected nodes

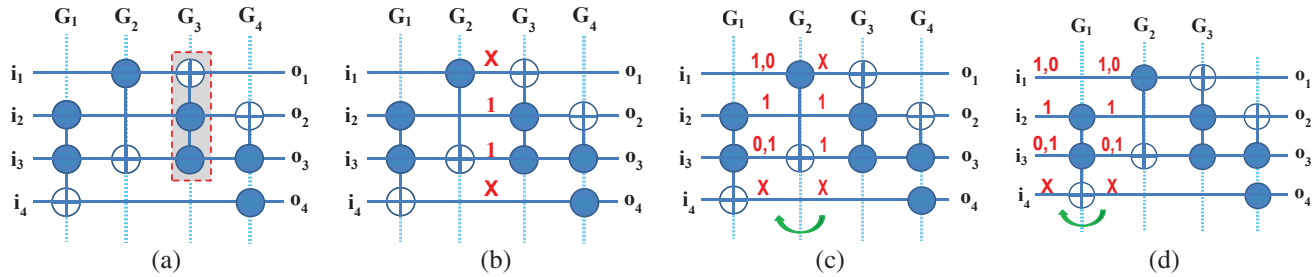


Fig. 4. Finding coverage list for each fault: (a) SMGF on G_3 (b) activating the fault by setting 1 to all control inputs of G_3 (c) and (d) back-tracing.

cover all connections to the fault nodes (all faults are detected). A portion of such graph has been shown in Fig. 5.

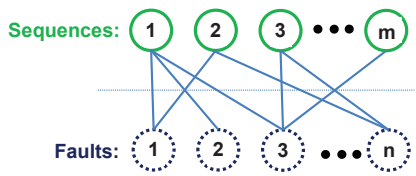


Fig. 5. A portion of a bipartite graph representing the sequences and the faults

Since minimum dominating set bipartite problem is a NP-hard problem, hence we have used a greedy algorithm to select the sequences for a Ping-Pong test. This algorithm uses a greedy approach to select the sequence nodes which cover all faults, with the minimum cost. The algorithm has the following steps:

1) Sort the sequence nodes based on the number of faults detected by each sequence.
2) Among the sequence nodes which have the maximum coverage, select the node with the minimum cost.
3) Remove all the fault nodes which are detected by the selected sequence node.
4) For each sequence node, update the cost and the number of detected faults, based on the remaining fault nodes.
5) Repeat the steps while there is at least one fault node.

A sequence may contain one cycle or more, without any loops. The chance of selection of a sequence is increased by its coverage for undetected faults. The computation complexity of this algorithm is proportional to the number gates and the number of sequences (Ping-Pong sessions).

E. Discussion

The presented implementation of Ping-Pong test consists of four steps. The computation complexity of this implementation is exponentially proportional to the number of inputs (which is typically small). Aforementioned implementation of the Ping-Pong test achieves 100% coverage in testing for single SMGFs and single RGFs. However, with minor adjustment, this method can be applied to other fault models. for this purpose, fault simulation and fault dropping need to be done for the new fault model (e.g. PMGF, MMGF). It must be noted, an SMGF (RGF) does not change a reversible circuit to an irreversible one. However, if any other fault or defect does such an effect, the proposed method can still detect the fault, as long as the fault is observable at the primary outputs.

V. Experimental Results

In order to evaluate the efficiency of the proposed Ping-Pong testing method, it is verified on a set of reversible benchmark circuits [23]. Our implemented flow extracts the required information for Ping-Pong testing of each benchmark circuit. This information includes the required sequences in the forms of $(V^1, k, f(V^k))$, where V^1 is the starting vector of the sequence, k indicates the number of cycles in the testing, and $f(V^k)$ indicates the expected output for the sequence.

The sequences are simulated with respects to different fault models. In each simulation, the fault is injected to the circuit, and then the sequences are applied to the faulty circuit. If the output of the faulty circuit for at least one of the sequences deviates from the expected output, then the fault is marked as detected.

The fault models used in these simulations include: *Repeated Gate Fault* (RGF), *Single Missing Gate Fault* (SMGF), and Multiple SMGF. In the fault simulation for single SMGF and RGF we applied all possible faults (exhaustive fault simulation). However, for the multiple SMGFs the faults are simulated for 10^6 random faulty cases. To simulate with respect to multiple SMGFs faults, the faults are injected to a set of gates which are randomly selected.

Table I shows the simulation results for some benchmark circuits. The test generation time, based on the presented algorithms, was reasonable for these circuits. Nevertheless, we plan to enhance the algorithms in our future work for better scalability for larger circuits. The first three columns of this table show the circuit names, the number of gates in the circuit (m), and the number of inputs (n). Extracted Ping-Pong sessions are shown in the last three columns, in terms of the number of sessions (N_S), the total number of test cycles (N_C), and the fault converge of the Ping-Pong sessions for multiple SMGF. Furthermore, this table also shows the number of required test vectors, generated by random test pattern generation as well as using ATPG (greedy and branch and bound algorithms) for some of the benchmark circuits [8]. It must be noted, required information to test the circuit is twice the number of test vectors for each circuit. As seen in the table, total number of test vectors required for Ping-Pong test (total number of test cycles) is almost the same as total number of test vectors generated using ATPG of [8].

TABLE I

NUMBER OF TEST VECTORS TO DETECT ALL SMGFS (RGFS): USING PING-PONG TEST, RANDOM TEST PATTERN GENERATION (Rnd), GREEDY (Gr) AND BRANCH AND BOUND (B&B) ATPGS [8]. THE NUMBER OF PING-PONG SESSIONS (N_S) AND TEST CYCLES (N_C) OF PING-PONG TEST.

	Circuit Characteristics		Test Generation [8]			Ping-Pong Test		
Circuit	m	n	Rnd	Gr	B&B	N_S	N_C	Multiple SMGF
$ham3tc$	5	3	2	2	2	2	1+2	80.02%
3_17tc	6	3	3	2	2	1	2	78.44%
$mod5d1$	8	5	3	1	1	1	1	47.09%
$2of5d2$	12	7	6	2	2	1	3	82.84%
$mod5adders$	17	6	5	3	3	2	2+1	86.25%
$5mod5tc$	17	6	6	1	1	1	1	48.63%
$ham15tc1$	70	15	17	7	>2	1	9	98.57%
$5mod5_10_10_71a$	10	6	-	-	-	1	2	64.65%
$mspk_nth_primes4_11$	11	4	-	-	-	1	4	96.92%
$mspk_4_49_14$	14	4	-	-	-	1	6	84.86%
$mspk_nth_primes4_14$	14	4	-	-	-	1	4	89.41%
$4b15g_3$	15	4	-	-	-	1	4	90.78%
$cycle10_2$	19	12	-	-	-	2	11+1	94.74%
$gf2\hat{}4mult_19_83$	19	12	-	-	-	1	1	88.89%
$ham7_21_69$	21	7	-	-	-	1	3	94.39%
$nth_prime5_inc_25_103$	25	5	-	-	-	1	4	93.84%
$mspk_hwb5_31_91_opt_38_80$	38	5	-	-	-	2	4+3	97.28%
$hwb6_47_107$	47	6	-	-	-	2	5+2	97.78%
$nth_prime6_inc_55_667$	55	6	-	-	-	2	26+7	99.98%
$nth_prime7_inc_1427_3172$	1427	7	-	-	-	3	19+7+1	99.99%
$nth_prime8_inc_3346_7618$	3346	8	-	-	-	2	36+37	99.99%

As seen in the table, the implementation of Ping-Pong test with maximum three sessions achieves 100% coverage for single SMGF as well as single RGF. For the multiple SMGFs, the testing results in 86% fault coverage, in average. However, the implementation can be modified to achieve full coverage for multiple SMGFs at the cost of extra cycles and test sessions.

VI. SUMMARY AND CONCLUSIONS

Reversible logic as an inherent design requirement of quantum computers is a promising computational approach to achieve ultra-low power computation. However, due to anticipated high fault rates for future technologies, testing is obviously an important reliability component for reversible circuits. In this paper, we present a new testing approach for reversible circuits, which requires a minimum amount of test information, and makes it suitable for BIST of reversible circuits. We have also proposed a test generation method to reduce test time while achieving the desired fault coverage. Simulation results on a set of reversible benchmark circuits confirm that this approach can detect all of the single missing/repeated gate faults in a circuit.

Our future work includes the reduction of time complexity of test sequence generation, evaluation of Ping-Pong test with respect to other fault models, and the implementation of Ping-Pong based reversible BIST circuitry.

REFERENCES

[1] C. Bennett and R. Landauer. The fundamental limits of computation. In *Sci. American*, 253 (1): 48–56. 1985.

[2] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. Smolin, and H. Weinfurter. Elementary gates for quantum computation. *Phy. Rev. A*, 52 (5): 3457-3467, 1995.

[3] Yao-Hsin Chou, Sy-Yen Kuo, and I-Ming Tsai. Qbist: Quantum built-in self-test for any boolean circuit. In *VTS'08*, 261–266, 2008.

[4] E. Knill, R. Laflamme, and G. J. Milburn. A scheme for efficient quantum computation with linear optics. *Nature*, 409: 46–52, 2001.

[5] K. N. Patel, J. P. Hayes, and I. L. Markov. Fault testing for reversible circuits. *TCAD*, 23(8): 1220–1230, 2004.

[6] K. N. Patel, J. P. Hayes, and I. L. Markov. Fault testing for reversible circuits. In *VTS'03*, 410–416, 2003.

[7] I. Polian, T. Fiehn, B. Becker, and J. P. Hayes. A family of logical fault models for reversible circuits. In *ATS'05*, 422–427, 2005.

[8] J. P. Hayes, I. Polian, and B. Becker. Testing for missing-gate faults in reversible circuits. In *ATS'04*, 100–105, 2004.

[9] W. Liu F. Xiao, H. Chen and Z. Li. Fault detection for single and multiple missing-gate faults in reversible circuits. In *CEC'08*, 131–135, 2008.

[10] D.K. Kole, H. Rahaman, D.K. Das, and B.B. Bhattacharya. Derivation of optimal test set for detection of multiple missing-gate faults in reversible circuits. In *ATS'10*, 33–38, 2010.

[11] R. Wille, M. Soeken, and R. Drechsler. Reducing the number of lines in reversible circuits. In *DAC'10*, 647–652, 2010.

[12] W.D. Pan and M. Nalasani. Reversible logic. *IEEE Potentials*, 24(1): 38–41, 2005.

[13] R. Feynman. Quantum mechanical computers. *Foundations of Physics*, 16: 507–531, 1986.

[14] E. Fredkin and T. Toffoli. Conservative logic. In *Int. J. of Theo. Phy.*, 21: 219–253, 1982.

[15] M.A. Nielsen and I.L. Chuang. Quantum computation and quantum information. *Cambridge Univ. Press*, 2000.

[16] Quantum Computation (QC) Roadmap. http://qist.lanl.gov/, 2004.

[17] I. Polian and J.P. Hayes. Advanced modeling of faults in reversible circuits. In *EWDTS'10*, 376–381, 2010.

[18] V. Iyengar, K. Chakrabarty, and B. T. Murray. Deterministic built-in pattern generation for sequential circuits. *Journal of Electronic Testing*, 15: 97–114, 1999.

[19] A. Chandra and K. Chakrabarty. Test data compression and decompression based on internal scan chains and golomb coding. *TCAD*, 21(6): 715 –722, 2002.

[20] P. H. Bardell. Self-testing of multi-chip logic modules. *ITC'82*, 200–204, 1982.

[21] A. Krasniewski and S. Pilarski. Circular self-test path: a low-cost bist technique for vlsi circuits. *TCAD*, 8(1): 46 –55, 1989.

[22] N.A. Touba. Circular *BIST* with state skipping. *TVLSI*, 10(5): 668 –672, 2002.

[23] Reversible logic synthesis benchmarks page: http:// webhome.cs.uvic.ca / $\sim$ dmaslov/.

[24] M. Liedloff. Finding a dominating set on bipartite graphs. *Inf. Process. Lett.*, 107: 154–157, 2008.